
```
from typing import Any
import datetime
spam: Any = 42
spam = datetime.date.today()
spam = True
```

In this example, the `Any` type hint allows you to set the `spam` variable to a value of any data type, such as `int`, `datetime.date`, or `bool`. You can also use `object` as the type hint, because this is the base class for all data types in Python. But `Any` is a more readily understandable type hint than `object`.

As you should with `Union` and `Optional`, use `Any` sparingly. If you set all of your variables, parameters, and return values to the `Any` type hint, you'd lose the type-checking benefits of static typing. The difference between specifying the `Any` type hint and specifying no type hint is that `Any` explicitly states that the variable or function accepts values of any type, whereas an absent type hint indicates that the variable or function has yet to be type hinted.

Setting Type Hints for Lists, Dictionaries, and More

Lists, dictionaries, tuples, sets, and other container data types can hold other values. If you specify `list` as the type hint for a variable, that variable must contain a list, but the list could contain values of any type. The following code won't cause any complaints from a type checker:

```
spam: list = [42, 'hello', 3.14, True]
```

To specifically declare the data types of the values inside the list, you must use the `typing` module's `List` type hint. Note that `List` has a capital `L`, distinguishing it from the `list` data type:

```
from typing import List, Union
❶ catNames: List[str] = ['Zophie', 'Simon', 'Pooka', 'Theodore']
❷ numbers: List[Union[int, float]] = [42, 3.14, 99.9, 86]
```

In this example, the `catNames` variable contains a list of strings, so after importing `List` from the `typing` module, we set the type hint to `List[str]` ❶. The type checker catches any call to the `append()` or `insert()` method, or any